

Autodesk® Scaleform®

文本繪製 API

Scaleform 4.2
GFX::Movie

DrawTextAPI
ActionScript

API

C++

Artem Bolgar

2.0

2011 4 22

Copyright Notice

Autodesk® Scaleform® 4.2

© 2012 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo) Built with ObjectARX (design/logo), Burn, Buzzsaw, CAiCE, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWFx, DXF, Ecotect, Evolver, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Scaleform GfX, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, Tinkerbox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

API

Autodesk Scaleform Corporation

6305 Ivy Lane, Suite 310

Greenbelt, MD 20770, USA

www.scaleform.com

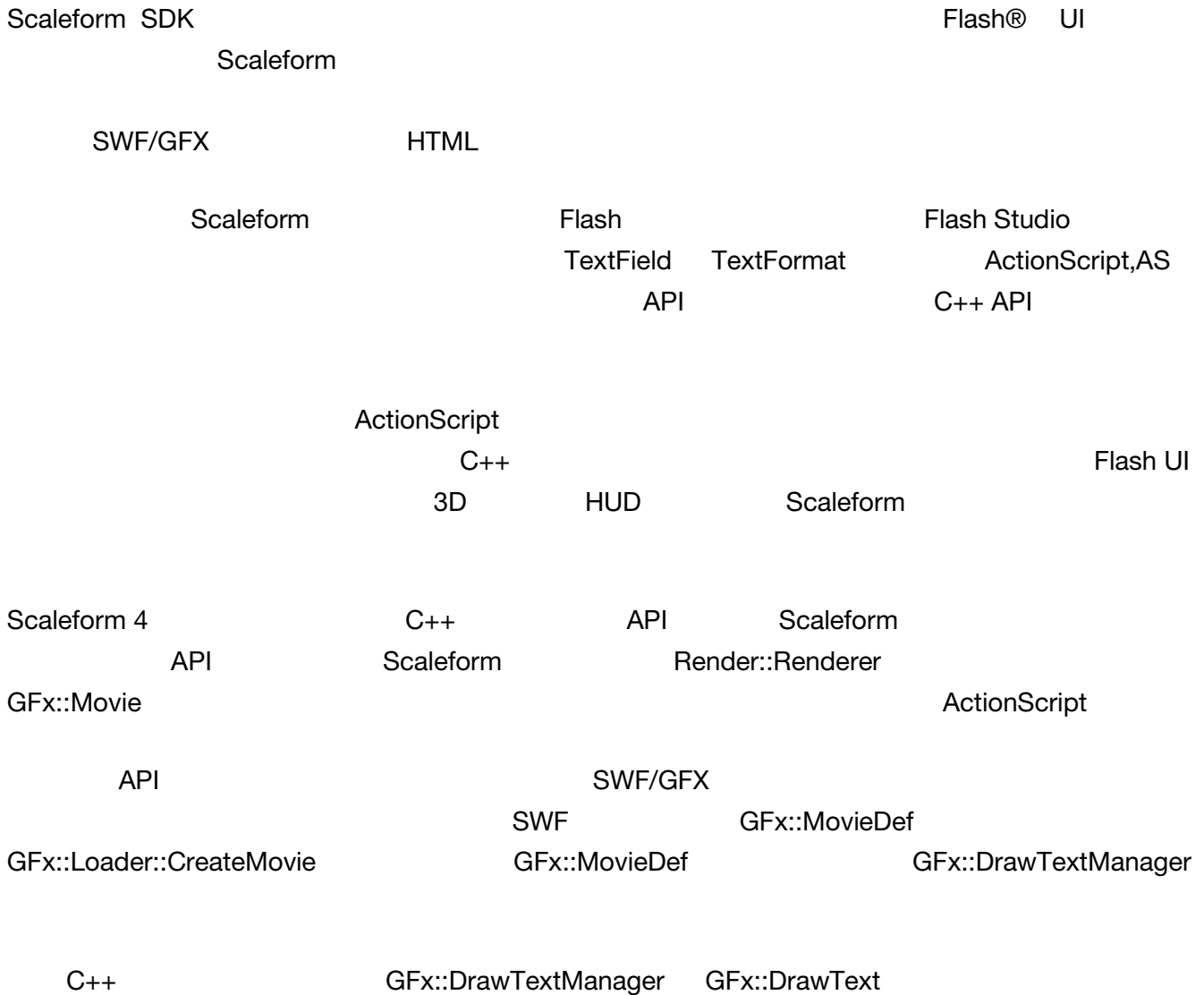
info@scaleform.com

(301) 446-3200

(301) 446-3199

1		1
1.1	DrawTextManager	2
1.1.1	DrawText	3
1.1.2	Rendering	5
1.1.3		5
1.2	DrawText	7
1.2.1		7
1.2.2	HTML	7
1.2.3		8
1.2.4		9
1.2.5		9
2	DrawText	11

1



- *DrawTextManager* – DrawText /

- *DrawText* – Flash HTML

SetColor, SetFont, SetFontStyle

1.1 DrawTextManager

DrawTextManager DrawText

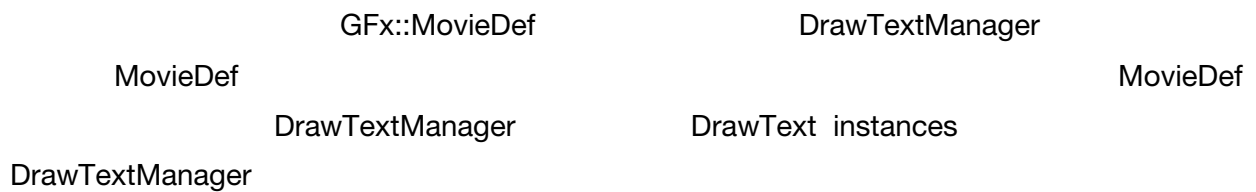
DrawTextManager

1. DrawTextManager();

SetFontProvider

```
Ptr<DrawTextManager> pdm = *new DrawTextManager();
```

2. DrawTextManager(MovieDef* pmovieDef);



```
Ptr<MovieDef> pmd = *loader.CreateMovie("drawtext fonts.swf");
Ptr<DrawTextManager> pdm = *new DrawTextManager(pmd);
```

3. DrawTextManager(Loader* ploader);

Gfx::Loader

```
Loader loader;
..
Ptr<DrawTextManager> pdm = *new DrawTextManager(&loader);
```

1.1.1 DrawText

DrawTextManager	DrawText	DrawTextManager
DrawText		

- DrawText* CreateText(const char* putf8Str, const RectF& viewRect, const TextParams* ptxtParams = NULL);
- DrawText* CreateText(const wchar_t* pwstr, const RectF& viewRect, const TextParams* ptxtParams = NULL);
- DrawText* CreateText(const String& str, const RectF& viewRect, const TextParams* ptxtParams = NULL);

	DrawText	UTF8
Unicode/UCS-2	Scaleform::String	
putf8str	UTF-8	
pwstr	Unicode/UCS-2	
str:	Scaleform::String.	

viewRect:

BeginDisplay

ptxtParams:	TextParams
-------------	------------

```
struct TextParams
{
    Color                TextColor;
    DrawText::Alignment  HAlignment;
    DrawText::VAlignment VAlignment;
    DrawText::FontStyle  FontStyle;
    float                FontSize;
    String               FontName;
    bool                 Underline;
    bool                 Multiline;
    bool                 WordWrap;
};
```

TextColor

HAlignment DrawText::Align_Left (), DrawText::Align_Right,
 DrawText::Align_Center, DrawText::Align_Justify

VAlignment : DrawText::VAlign_Top (), DrawText::VAlign_Bottom,
 DrawText::VAlign_Center

FontStyle DrawText::Normal,
 DrawText::Bold, DrawText::Italic, DrawText::BoldItalic (DrawText::ItalicBold)

FontSize

FontName: Arial Times New Roman Bold
 Italic

Underline

Multiline: /

WordWrap /

ptxtParams DrawTextManager

```

void SetDefaultTextParams(const TextParams& params);
const TextParams& GetDefaultTextParams() const;

```

DrawTextManager - CreateHtmlText CreateText
 HTML DrawText

```

// HTML DrawText
DrawText* CreateHtmlText(const char* putf8Str, const RectF& viewRect);
DrawText* CreateHtmlText(const wchar_t* pwstr, const RectF& viewRect);
DrawText* CreateHtmlText(const String& str, const RectF& viewRect);

```

```

// Gfx::DrawText
String str("String No 2");
DrawTextManager::TextParams params;
params.FontName = "Symbol";

```



```

params.FontSize    = 30;
params.FontStyle   = DrawText::Italic;
params.Multiline   = false;
params.HAlignment  = DrawText::Align_Right;
params.VAlignment  = DrawText::VAlign_Bottom;

Ptr<DrawText> ptxt;
ptxt = *pdm->CreateText(str, RectF(20, 300, 400, 700), &params);

// HTML   GFx::DrawText
Ptr<DrawText> ptxt;
ptxt = *pdm->CreateHtmlText("<p><FONT size='20'>AB
                             <b>singleline</b><i> CD</i>0",
                             RectF(20, 300, 400, 700));

```

1.1.2 Rendering

GFx::Movie		DisplayHandle (GFx::DrawTextManager)
(DrawTextManager::SetViewport)	“ ”	(Viewport) GFx

1.1.3

DrawTextManager

```

SizeF GetTextExtent(const char* putf8Str, float width = 0,
                    const TextParams* ptxtParams = 0);
SizeF GetTextExtent(const wchar_t* pwstr, float width = 0,
                    const TextParams* ptxtParams = 0);
SizeF GetTextExtent(const String& str, float width = 0,
                    const TextParams* ptxtParams = 0);

SizeF GetHtmlTextExtent(const char* putf8Str, float width = 0,
                        const TextParams* ptxtParams = 0);
SizeF GetHtmlTextExtent(const wchar_t* pwstr, float width = 0,
                        const TextParams* ptxtParams = 0);
SizeF GetHtmlTextExtent(const String& str, float width = 0,
                        const TextParams* ptxtParams = 0);

```

GetTextExtent

TextParams

GetHtmlTextExtent

HTML

TextParams

width

ptxtParams

SetDefaultTextParams/GetDefaultTextParams

HTML

ptxtParams

HTML

ptxtParams

```
//
String str("String No 2");
DrawTextManager::TextParams params;
params.FontName    = "Symbol";
params.FontSize    = 30;
params.FontStyle   = DrawText::Italic;
params.Multiline   = false;
params.HAlignment  = DrawText::Align_Right;
params.VAlignment  = DrawText::VAlign_Bottom;

SizeF sz = pdm->GetTextExtent(str, 0, params);

params.WordWrap    = true;
params.Multiline   = true;
sz = pdm->GetTextExtent(str, 120, params);

// HTML
const wchar_t* html =
    L"<p><FONT size='20'>AB <b>singleline</b><i> CD</i>0";
sz = pdm->GetHtmlTextExtent(html);

sz = pdm->GetHtmlTextExtent(html, 150);
```

1.2 DrawText

DrawText HTML

1.2.1

SetText	UTF8	UCS2	Scaleform::String
lengthInBytes'		UTF8	lengthInChars'
	UTF-8	UCS-2	

```
void SetText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));
void SetText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));
void SetText(const String& str);
```

GetText UTF8 HTML
HTML

```
String GetText() const;
```

1.2.2 HTML

SetHtmlText	UTF8,UCS2	Scaleform::String HTML	HTML
lengthInBytes'	UTF8	lengthInChars'	
	UTF-8	UCS-2	

```
void SetHtmlText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));
void SetHtmlText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));
void SetHtmlText(const String& str);
```

GetHtmlText HTML
SetColor, SetFont HTML

```
String GetHtmlText() const;
```

1.2.3

```
void SetColor(Color c, UPInt startPos = 0, UPInt endPos = UPInt(-1));
                        [startPos..endPos]                (R, G, B, A)

void SetFont (const char* pfontName, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                        [startPos..endPos]                startPos      endPos

void SetFontSize(Float fontSize, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                        [startPos..endPos]                startPos      endPos '

void SetFontStyle(FontStyle, UPInt startPos = 0, UPInt endPos = UPInt(-1));

enum FontStyle
{
    Normal,
    Bold,
    Italic,
    BoldItalic,
    ItalicBold = BoldItalic
};

                        [startPos..endPos]                startPos
                        endPos

void SetUnderline(bool underline, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                        [startPos..endPos]

void SetMultiline(bool multiline);
                        multiline      true      false

bool IsMultiline() const;
                        true      false

void SetWordWrap(bool wordWrap);
/

bool Wrap() const;
```

1.2.4

```
Alignment    Valignment
enum Alignment
{
    Align_Left,
    Align_Default = Align_Left,
    Align_Right,
    Align_Center,
    Align_Justify
};
enum Valignment
{
    VAlign_Top,
    VAlign_Default = VAlign_Top,
    VAlign_Center,
    VAlign_Bottom
};
```

```
void SetAlignment(Alignment);
```

```
Alignment    GetAlignment() const;
```

```
void SetValignment(Valignment);
```

```
Valignment    GetValignment() const;
```

1.2.5

```
DrawText
```

```
void SetRect(const RectF& viewRect);
```

```
RectF GetRect() const;
```

```
void SetMatrix(const Matrix& matrix);
```

```
const Matrix GetMatrix() const;
```

```
void SetCxform(const Cxform& cx);
```

```
const Cxform& GetCxform() const;
```

2 DrawText

Scaleform 3.3	DrawText API	Scaleform SDK
---------------	--------------	---------------

DrawTextAPI

DrawText API

FxPlayerApp

FxPlayerApp FxPlayerAppBase

FxPlayerApp OnInit DrawText OnUpdateFrame


```

class FxPlayerApp : public FxPlayerAppBase
{
    public:
        FxPlayerApp();

        virtual bool          OnInit(Platform::ViewConfig& config);
        virtual void          OnUpdateFrame(bool needRepaint);

        Ptr<GFx::DrawTextManager> pDm1, pDm2;
        Ptr<GFx::DrawText>        ptxt11, ptxt12, ptxt21, ptxt22, ptxtImg,
                                pblurTxt, pglowTxt, pdropShTxt, pblurglowTxt;

        float   Angle;
        UInt32  Color;
};

```

DrawText DrawTextManager

1

DrawTextManager GFx::Loader


```

pDm1 = *new DrawTextManager(&mLoader);

```


GFx::FontProvider FontProvider

FxPlayerAppBase::OnInit

```

pDm1->SetFontProvider(mLoader.GetFontProvider());

```

API

HTML

DrawText

Scaleform::String

DrawText

//

DrawTextManager::TextParams defParams =

pDm1->GetDefaultTextParams();

defParams.TextColor = Color(0xF0, 0, 0, 0xFF); // red, alpha = 255

defParams.FontName = "Arial";

defParams.FontSize = 16;

pDm1->SetDefaultTextParams(defParams);

CreateText

SetDefaultTextParams

```
ptxt11 = *pDm1->CreateText ("Plain text, red, Arial,
                             16pts",RectF(20, 20, 500, 400));
```

DrawTextManager

GetTextExtent

DrawText

// Scaleform::String TextParams

String str(

```
"Scaleform Gfx is a light-weight high-performance rich media vector
graphics and user interface (UI) engine.");
```

Gfx::DrawTextManager::TextParams params;

params.FontName = "Arial";

params.FontSize = 14;

params.FontStyle = DrawText::Italic;

params.Multiline = true;

params.WordWrap = true;

params.HAlignment = DrawText::Align_Justify;

sz = pDm1->GetTextExtent(str, 200, &params);

ptxt12 = *pDm1->CreateText(str, RectF(200, 300, sz), &params);

ptxt12->SetColor(Render::Color(0, 0, 255, 130), 0, 1);

DrawText

DrawText

DrawText

DisplayHandle

RenderThread:

Render::Size<unsigned> viewSize = GetViewSize();

Render::Viewport dmViewport(viewSize.Width, viewSize.Height,

int(viewSize.Width * GetSafeArea().Width),

int(viewSize.Height * GetSafeArea().Height),

int(viewSize.Width - 2 * viewSize.Width * GetSafeArea().Width),

int(viewSize.Height - 2 * viewSize.Height * GetSafeArea().Height));


```

    pDm1->SetViewport(dmViewport);
    pDm1->Capture();
    pRenderThread->AddDisplayHandle(pDm1->GetDisplayHandle(),
                                    FxRenderThread::DHCAT_Overlay, false);

```

2

	DrawTextManger	Gfx::MovieDef	MovieDef
MovieDef	SWF	Scaleform	Integration Tutorial
Scaleform			

```

Ptr<MovieDef> pmovieDef = *mLoader.CreateMovie("drawtext_fonts.swf",
                                                Loader::LoadAll);
pDm2 = *new DrawTextManager(pmovieDef);

```

HTML HTML **DrawTextManager::GetHtmlExtent method**

```

//      HTML      Gfx::MovieDef
const wchar_t* html = L"<P>o123 <FONT FACE=\"Times New Roman\" SIZE
                        =\"140\">L\"A<b><i><FONT
COLOR='#3484AA'>б</FONT></i>пак</b>адабpA!</FONT></P>"
L"<P><FONT FACE='Arial Unicode MS'>Hànyǔ;    /    </FONT></P>"
L"<P><FONT FACE='Batang'>    /    </FONT></P>"
L"<P><FONT FACE='Symbol'>Privet!</FONT></P>";

Gfx::DrawTextManager::TextParams defParams2 = pDm2->GetDefaultTextParams();
defParams2.TextColor = Color(0xF0,0,0,0xFF); //red,alpha = 255
defParams2.Multiline = true;
defParams2.WordWrap = false;
SizeF htmlSz = pDm2->GetHtmlTextExtent(HtmlText, 0, &defParams2);
ptxt22 = *pDm2->CreateHtmlText(HtmlText, RectF(00, 100, htmlSz),
&defParams2);

SizeF sz;
sz = pDm2->GetTextExtent("Animated");
ptxt21 = *pDm2->CreateText("Animated", RectF(600, 400, sz));
ptxt21->SetColor(Render::Color(0, 0, 255, 255));
Angle = 0;

```

:

```

SizeF sz;
Ptr<GFx::DrawText> pglowTxt
pglowTxt = *pDm2->CreateText("Glow", RectF(800, 350, SizeF(200, 220)));
pglowTxt->SetColor(Render::Color(120, 30, 192, 255));
pglowTxt->SetFontSize(72);
GFx::DrawText::Filter glowF(GFx::DrawText::Filter_Glow);
glowF.Glow.BlurX = glowF.Glow.BlurY = 2;
glowF.Glow.Strength = 1000;
glowF.Glow.Color = Render::Color(0, 0, 0, 255).ToColor32();
pglowTxt->SetFilters(&glowF);

```

DrawText	DrawText	DrawText DisplayHandle
RenderThread		

```

pDm2->SetViewport(dmViewport);
pDm2->Capture();
pRenderThread->AddDisplayHandle(pDm2->GetDisplayHandle(),
                                FxRenderThread::DHCAT_Overlay, false);

```

DrawText **SetMatrix** **SetCxfom**

FxPlayerApp::OnUpdateFrame

```

void FxPlayerApp::OnUpdateFrame( bool needRepaint )
{
    DrawText::Matrix txt21matrix;
    Angle += 1;
    RectF r = ptxt21->GetRect();
    txt21matrix.AppendTranslation(-r.x1, -r.y1);
    txt21matrix.AppendRotation(Angle*3.14159f / 180.f);
    txt21matrix.AppendScaling(2);
    txt21matrix.AppendTranslation(r.x1, r.y1);
    ptxt21->SetMatrix(txt21matrix);

    pDm2->Capture();

    FxPlayerAppBase::OnUpdateFrame(needRepaint);
}

```

Bin/Samples

drawtext_fonts.swf

Projects/Win32/Msvc90/Samples/DrawText

Visual Studio

2008

